CFL1 - Solution Sketches

February 21, 2023

We sketch here the solutions for the exercises of lecture CFL1 in a brief manner. Note that a proper solution would require more detailed descriptions, explanations, and in some cases examples. Some of the exercises may have more than one solution, and we just show one of them. When the exercise requests a context-free grammar, we provide one in a very compact way (as done in the slides) where we just provide the productions, with the implicit assumption that non-terminals start with capital letters, and that the initial symbol is the on corresponding to the first production.

Exercise 1.1.(a)

$$S \quad \rightarrow \quad \epsilon \mid 0S$$

To actually convince us that the above grammar G is a correct solution we need to prove that its language $L(G)$ coincides with $L = \{0^n \mid n \geq 0\}$. We can do this by proving that for every word w (1) $S \in L(G)$ implies $w \in L$ and (2) $w \in L$ implies $w \in L(G)$.

We show (1) by induction and using the bottom-up definition of a language (recursive inference). The base case is the production $S \rightarrow \epsilon$, from which we have that $\epsilon \in L(G)$. Trivially ϵ belongs also to L.

For the inductive step, we consider the production $S \rightarrow 0S$, from which we have that if a word $w' \in L(G)$ then $0w' \in L(G)$. Using the induction hypothesis $w' \in L(G)$ implies $w' \in L$. All we need to prove is that if $w' \in L$ then $0w' \in L$ which is trivial.

To show (2) we can proceed by induction on the length of $w = 0^n$. As base case take $n = 0$. In this case $w = \epsilon$. By using production $S \rightarrow \epsilon$ we know that $\epsilon \in L(G)$.

For the inductive step, we need to show that $w = 0^{n+1} \in L(G)$. By the inductive hypothesis we know that $0^n \in L(G)$. We can the use production $S \rightarrow 0S$ to conclude that $00^n = 0^{n+1}$ belongs to $L(G)$.

Exercise 1.1.(b)

$$S \rightarrow 0 \mid 0S$$

Exercise 1.1.(c)

$$\begin{array}{lcl} S & \rightarrow & AB \\ A & \rightarrow & 0 \mid 0A \\ B & \rightarrow & 1 \mid 1B \end{array}$$

Exercise 1.1.(d)

$$S \rightarrow 01 \mid 0S1$$

We can prove correctness of this solution by showing that the language $L = \{0^n 1^n \mid n \geq 1\}$ and the language L_S defined by S in our CFG is the same.

To do this we will prove two statements:

(1) For any word w, if $w \in L$ then $w \in L_S$

(2) For any word w, if $w \in L_S$ then $w \in L_1$

Proving (1) To prove (1) we observe that all words in S are of the form $0^n 1^n$. We can then prove the statement by induction on n.

Base case: $n = 1$. We have then that $w = 0^1 1^1 = 01$. We can show that $01 \in L_S$ with the simple derivation $S \Rightarrow 01$ obtained by applying the production $S \rightarrow 01$.

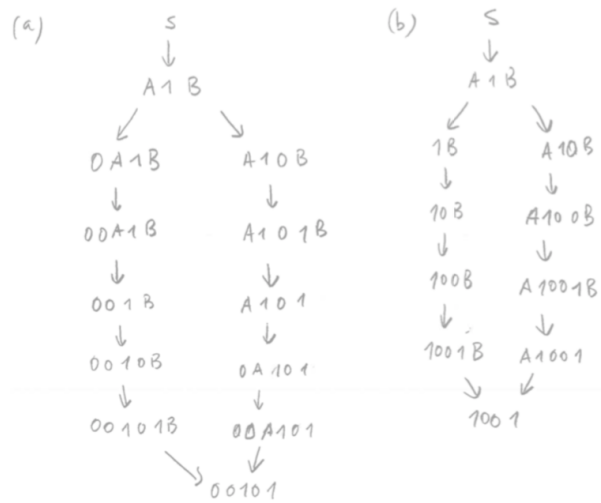
Inductive case: We will assume that (1) holds for words of the form $0^n 1^n$ and we will show that it then holds for words of the form $w = 0^{n+1} 1^{n+1}$. It is trivial to see that w can be rewritten as follows $w = 0^{n+1} 1^{n+1} = 00^n 1^n 1$. Now, by applying the induction hypothesis we know that $0^n 1^n \in L_S$. We can then use production $S \rightarrow 0S1$ and inference, to conclude that $00^n 1^n 1 \in L_S$.

To prove (2) we can apply induction on the length of the derivation or inference used to show that w is in L_S.

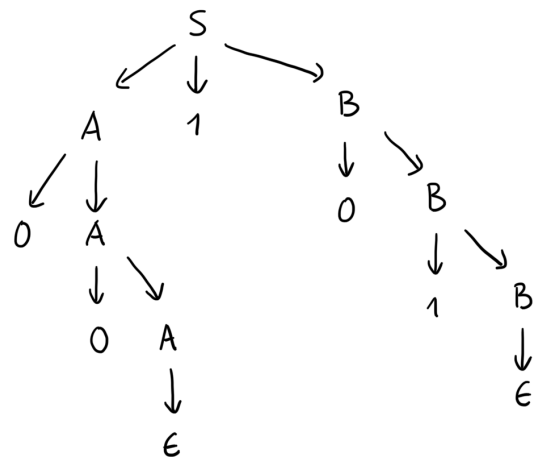
Base case: The only base case for inferring that $w \in L_S$ is when w has been obtained with the production $S \rightarrow 01$. In this case $w = 01$, which is clearly in L_S (in particular, for $n = 1$).

Inductive case. Assume that (2) holds for any word w. We have show that for any word w' that we can infer to be in the language with just one production, then w' is in L. The only inference step is provided by using production $S \rightarrow 0S1$. This yields $w' = 0w1$. By the induction hypothesis, we know that $w = 0^n 1^n$ for some number $n \geq 1$. It is trivial to see that we can rewrite w' as follows: $w' = 00^n 1^n 1 = 0^{n+1} 1^{n+1}$. Hence, $w' \in L$.

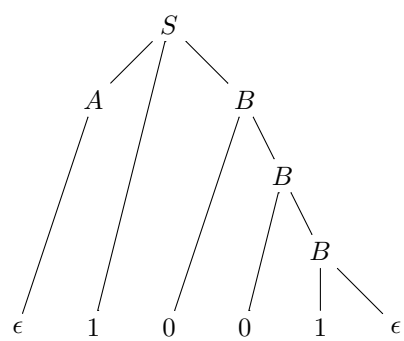
Exercise 1.2. For simplicity, the left- and right-most derivations are drawn in the same picture below.


(a)
S
A1B
0A1B
A10B
00A1B
A101B
001B
A101
0010B
0A101
00101B
00A101
00101
(b)
S
A1B
1B
A10B
10B
A100B
100B
A1001B
1001B
A1001
1001

Exercise 1.3.(a)



Exercise 1.3.(b)



Exercise 1.4.(b)

$$S \rightarrow \epsilon \mid SS \mid (S)$$

Exercise 1.4.(c)

$$S \rightarrow 010 \mid 0S0$$

Exercise 1.4.(d)

$$\begin{aligned} S &\rightarrow 0S2 \mid A \\ A &\rightarrow \epsilon \mid 1A \end{aligned}$$

Exercise 1.4.(e)

$$\begin{aligned} S &\rightarrow AB \\ A &\rightarrow \epsilon \mid 0A1 \\ B &\rightarrow \epsilon \mid 1B \end{aligned}$$

Exercise 1.4.(f)

$$S \rightarrow 011 \mid 0S11$$

Exercise 1.5. We could start with simple lists where symbols are just put one after the other, as in "010101010101". For lists of such format without separators the grammar is simple:

$$\begin{array}{lcl} S & \rightarrow & \epsilon \mid AS \\ A & \rightarrow & 0 \mid 1 \end{array}$$

If we want to admit separators (for example, "," as in "0,1,0,1,0,1,0,1,0,1,0,1") we need a different grammar:

$$\begin{array}{lcl} S & \rightarrow & \epsilon \mid N \\ N & \rightarrow & A \mid A, N \\ A & \rightarrow & 0 \mid 1 \end{array}$$

intuitively, N represents non-empty lists, and S possibly empty lists.

Exercise 1.6. In this solution we want to represent a simple tree with just one leaf 0 as the string 0. Composing two such leafs in a binary tree will be represented as $[0 \bullet 0]$. A complete binary tree of depth 2 would look like this $[[0 \bullet 0] \bullet [0 \bullet 0]]$. The following grammar accomplished this goal:

$$\begin{array}{lcll} T & \rightarrow & \epsilon & \text{(an empty tree)} \\ & \mid & A & \text{(a leaf)} \\ & \mid & [T \bullet T] & \text{(a root } \bullet \text{ with two subtrees)} \\ A & \rightarrow & 0 \mid 1 & \end{array}$$

Exercise 1.7.a Yes. Any mapping can be write down as a list of (key,value) "pairs" in the format "key $\mapsto$ value".

Exercise 1.7.b It depends on the semantic interpretation of the words. For example, consider the following word:

$$0 \mapsto a, 0 \mapsto b$$

There are several ways of interpreting it as a map: as an ill-formed one (because the value of key 0 is not well-defined), as the map $0 \mapsto b$ (if we interpret the list as updates on a map, overwriting previous maps), or as the map $0 \mapsto a$ (if we discard overwriting already defined pairs of key/value).

Exercise 1.7.c It depends on the semantic interpretation of the words, but for a reasonable interpretation the answer is yes. Here are some examples

$$0 \mapsto a, 1 \mapsto b$$

and

$$1 \mapsto b, 0 \mapsto a$$

denote the same map (the order of single maps is irrelevant).

Also $0 \mapsto a$, and $0 \mapsto a, \mathsf{nil}$, and $0 \mapsto a, \mathsf{nil}, \mathsf{nil}$, etc. denote the same map (since nil is irrelevant).

Exercise 1.8. We actually saw a very similar grammar in the RL part of the course:

$$E \quad \rightarrow \quad e \mid \emptyset \mid 0 \mid 1 \mid E + E \mid EE \mid E* \mid (E)$$

Exercise 1.9. We define a function cfg that, given a regular expression E (i.e. a word recognised by the grammar of Exercise 1.8), and a non-terminal symbol S provides a set of productions such such that the language defined by S is the same as the language of E. The function is defined by induction on the structure of E:

$$\begin{array}{rcl}
\mathsf{cfg}(e, S) & = & \{S \rightarrow \epsilon\} \\
\mathsf{cfg}(\emptyset, S) & = & \emptyset \\
\mathsf{cfg}(0, S) & = & \{S \rightarrow 0\} \\
\mathsf{cfg}(1, S) & = & \{S \rightarrow 1\} \\
\mathsf{cfg}(E + E', S) & = & \{S \rightarrow S_E \ , S \rightarrow \ S_{E'}\} \ \cup \ \mathsf{cfg}(E, S_E) \cup \mathsf{cfg}(E', S_{E'}) \\
\mathsf{cfg}(EE', S) & = & \{S \rightarrow S_E S_{E'}\} \ \cup \ \mathsf{cfg}(E, S_E) \ \cup \ \mathsf{cfg}(E', S_{E'}) \\
\mathsf{cfg}(E*, S) & = & \{S \rightarrow \epsilon, S \rightarrow S_E S\} \ \cup \ \mathsf{cfg}(E, S_E) \\
\mathsf{cfg}((E), S) & = & \{S \rightarrow S_E\} \ \cup \ \mathsf{cfg}(E, S_E)
\end{array}$$

NOTE: The set of terminal and non-terminal symbols and the initial symbols are implicit and can be reconstructed from the resulting productions and from the invocation of the function.

NOTE: If an expression E appears multiple times as a subexpression, the final result will not contain multiple times the same production, as those are combined with set union.

The correctness of the solution can be shown by structural induction, with base cases e, $\emptyset$, 0 and 1 and inductive cases for the rest of the equations above.